String Operators

In JavaScript, the + operator is used for string concatenation. If one of the operands is a string, JavaScript will automatically convert the other operand into a string and concatenate them. This operator always creates a new string.

javascript

Copy code

```
let greetings = "Hi";

console.log(greetings + " " + "Alice"); // -> Hi Alice


let sentence = "Happy New Year ";

let newSentence = sentence + 10191;


console.log(newSentence); // -> Happy New Year 10191

console.log(typeof newSentence); // -> string
```

Compound Assignment Operators

The compound assignment operator += is used to append values to a variable.

javascript

Copy code

```
let sentence = "Happy New ";

sentence += "Year ";

sentence += 10191;

console.log(sentence); // -> Happy New Year 10191
```

Comparison Operators

Comparison operators check the equality or inequality of values, returning true or false. These operators attempt to convert operands to a common type when necessary, except when comparing strings or using the identity operator ===.

- **Strict Equality (===)**: Checks if values are equal and of the same type.
- **Equality (==)**: Checks if values are equal but allows for type coercion.

javascript

Copy code

```
console.log(10 === 5); // -> false

console.log(10 === 10); // -> true

console.log(10 === 10n); // -> false

console.log(10 === "10"); // -> false

console.log("10" === "10"); // -> true

console.log(0 === false); // -> false


console.log(10 == 5); // -> false

console.log(10 == "10"); // -> true

console.log("10" == "10"); // -> true

console.log(0 == false); // -> true

console.log(NaN == NaN); // -> false
```

Non-Equality Operators

- **Non-strict Inequality (!==)**: Returns true if operands are not identical or of different types.
- **Inequality (!=)**: Returns true if operands are not equal (after type conversion).

javascript

Copy code

```
console.log(10 !== 5); // -> true

console.log(10 !== "10"); // -> true

console.log(10 != "10"); // -> false

console.log("Alice" != "Bob"); // -> true
```

Comparison Operators for Numerical Values

These operators check whether one operand is greater than, smaller than, or equal to another operand. They work well with numbers or values that can be converted to numbers.

javascript

Copy code

```
console.log(10 > 100); // -> false

console.log(101 > 100); // -> true

console.log("10" < 20n); // -> true
```

```
console.log(101 <= 100); // -> false

console.log(10 >= 10n); // -> true
```

String Comparison

Strings are compared character by character, using Unicode values. This comparison is case-sensitive and respects the order of characters in the alphabet.

javascript

Copy code

```
console.log("b" > "a"); // -> true

console.log("ab1" < "ab4"); // -> true

console.log("abA" > "ab4"); // -> true
```

Other Operators

typeof

The typeof operator is used to check the type of a variable or literal. It returns a string that represents the type.

javascript

Copy code

```
let year = 10191;

console.log(typeof year); // -> number

console.log(typeof false); // -> boolean
```

instanceof

The instanceof operator checks whether an object is an instance of a given class or prototype.

javascript

Copy code

```
let names = ["Patti", "Bob"];

console.log(names instanceof Array); // -> true
```

delete

The delete operator removes properties from an object.

javascript

Copy code

```
let user = { name: "Alice", age: 38 };

console.log(user.age); // -> 38

delete user.age;

console.log(user.age); // -> undefined
```

Ternary Operator

The ternary operator is a conditional operator that selects one of two values based on a condition.

javascript

Copy code

```
console.log(true ? "Alice" : "Bob"); // -> Alice

console.log(false ? "Alice" : "Bob"); // -> Bob
```

Precedence and Associativity

Operator precedence determines the order in which operators are evaluated in an expression. Some operators have higher precedence, and operations with the same precedence are evaluated according to their associativity (left to right or right to left).

javascript

Copy code

```
let a = 10;

let b = a + 2 * 3;

console.log(b); // -> 16
```

Operator Precedence Table (Partial)

The precedence table lists operators from highest to lowest precedence:

- **Grouping**: () (highest precedence)
- **Field access**: . (accessing object properties)
- **Function call**: () (function invocation)
- **Multiplication/Division**: * / %
- **Addition/Subtraction**: + -
- **Equality and Relational Operators**: == != === !== < > <= >=

This order affects how expressions are evaluated, and you can use parentheses to control the evaluation order explicitly.